

AI ASSISTED CODING

ASSIGNMENT 12.5

NAME: G.V.N HASINI

HTNO: 2303A51099

BATCH: 02

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - Ask AI to include time complexity and space complexity in the function docstring.
 - Verify the generated code with test cases.
- Expected Output:
 - A functional Python script implementing Merge Sort with proper documentation.

Code :

merge.py X

week 12 > merge.py > merge_sort

```
1  #write a python program to create a function merge_sort(arr) that sorts a list in
2  from heapq import merge
3  def merge_sort(arr):
4      """
5      Sorts a list in ascending order using the merge sort algorithm.
6
7      Time Complexity:  $O(n \log n)$  - where n is the number of elements in the list.
8      Space Complexity:  $O(n)$  - due to the temporary arrays used during merging.
9
10     Parameters:
11     arr (list): The list to be sorted.
12
13     Returns:
14     list: A new sorted list in ascending order.
15     """
16     if len(arr) <= 1:
17         return arr
18
19     mid = len(arr) // 2
20     left_half = merge_sort(arr[:mid])
21     right_half = merge_sort(arr[mid:])
22
23     return merge(left_half, right_half)
```

week 12 > merge.py > ...

```
23     return merge(left_half, right_half)
24 def merge(left, right):
25     result = []
26     i = j = 0
27
28     while i < len(left) and j < len(right):
29         if left[i] < right[j]:
30             result.append(left[i])
31             i += 1
32         else:
33             result.append(right[j])
34             j += 1
35
36     result.extend(left[i:])
37     result.extend(right[j:])
38
39     return result
```

```

week 12 > merge.py > ...
39     return result
40     # Test cases
41     if __name__ == "__main__":
42         test_cases = [
43             [38, 27, 43, 3, 9, 82, 10],
44             [5, 2, 9, 1, 5, 6],
45             [12, 11, 13, 5, 6, 7],
46             [],
47             [1],
48             [2, 1]
49         ]
50
51         for i, test in enumerate(test_cases):
52             sorted_list = merge_sort(test)
53             print(f"Test case {i + 1}: {sorted_list}")

```

Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python313\python.exe "c:\
.5.py"
Test Case 1: Input: [38, 27, 43, 3, 9, 82, 10] -> Sorted Output: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: Input: [5, 2, 9, 1, 5, 6] -> Sorted Output: [1, 2, 5, 5, 6, 9]
Test Case 3: Input: [] -> Sorted Output: []
Test Case 4: Input: [1] -> Sorted Output: [1]
Test Case 5: Input: [2, 1] -> Sorted Output: [1, 2]
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - Include docstrings explaining best, average, and worst-case complexities.
 - Test with various inputs.
- Expected Output:
 - Python code implementing binary search with AI-generated comments and docstrings.

Code :

week 12 > binary.py > ...

```
1 #write a python program to create a function binary_search(arr, target) returning the index of the
2 def binary_search(arr, target):
3     """
4     Performs binary search on a sorted list to find the index of the target element.
5
6     Time Complexity:
7     - Best Case:  $O(1)$  - when the target is at the middle of the list.
8     - Average Case:  $O(\log n)$  - where n is the number of elements in the list.
9     - Worst Case:  $O(\log n)$  - when the target is not present in the list or is at one of the ends.
10
11     Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.
12
13     Parameters:
14     arr (list): A sorted list in which to search for the target.
15     target: The element to search for.
16
17     Returns:
18     int: The index of the target if found, otherwise -1.
19     """
```

```
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Output :

```
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python38\python.exe .5.py
Test Case 1: Input: [1, 2, 3, 4, 5], Target: 3 -> Output: 2
Test Case 2: Input: [1, 2, 3, 4, 5], Target: 6 -> Output: -1
Test Case 3: Input: [], Target: 1 -> Output: -1
Test Case 4: Input: [1], Target: 1 -> Output: 0
Test Case 5: Input: [1, 2], Target: 2 -> Output: 1
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Code :

```
week 12 > smart healthcare.py >...
3  def sort_appointments_based_on_time_or_consultation_fee:
4  def search_appointment(appointments, appointment_id):
5      """
6      Searches for an appointment by its ID.
7
8      Time Complexity: O(n) - where n is the number of appointments.
9      Space Complexity: O(1) - as we are using a constant amount of space for variables.
10
11     Parameters:
12     appointments (list): A list of appointment records (dictionaries).
13     appointment_id (str): The ID of the appointment to search for.
14
15     Returns:
16     dict: The appointment record if found, otherwise None.
17     """
18     for appointment in appointments:
19         if appointment['appointment_id'] == appointment_id:
20             return appointment
21     return None
```

```

week 12 > smart healthcare.py > ...
22 def sort_appointments(appointments, key):
23     """
24     Sorts appointments based on a specified key (either 'time' or 'fee').
25
26     Time Complexity: O(n log n) - where n is the number of appointments.
27     Space Complexity: O(1) - as we are using a constant amount of space for variables.
28
29     Parameters:
30     appointments (list): A list of appointment records (dictionaries).
31     key (str): The key to sort by ('time' or 'fee').
32
33     Returns:
34     list: The sorted list of appointment records.
35     """
36     return sorted(appointments, key=lambda x: x[key])
37
38 # Test cases
39 if __name__ == "__main__":
40     appointments = [
41         {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'fee': 100},
42         {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'fee': 120},
43         {'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Johnson', 'time': '2024-07-01 09:30', 'fee': 150},
44     ]
45
46     # Test search_appointment
47     print(search_appointment(appointments, 'A002')) # Should return the appointment record for Jane Doe
48     print(search_appointment(appointments, 'A004')) # Should return None
49
50     # Test sort_appointments by time
51     sorted_by_time = sort_appointments(appointments, key='time')
52     print("Appointments sorted by time:")
53     for appointment in sorted_by_time:
54         print(appointment)
55
56     # Test sort_appointments by fee
57     sorted_by_fee = sort_appointments(appointments, key='fee')
58     print("Appointments sorted by fee:")
59     for appointment in sorted_by_fee:
60         print(appointment)

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python313\python.exe .5.py"
Found appointment: Bob with Dr. Jones at 2023-10-01 10:00:00 for $150

Appointments sorted by time:
Alice with Dr. Smith at 2023-10-01 09:00:00 for $100
Bob with Dr. Jones at 2023-10-01 10:00:00 for $150
Charlie with Dr. Brown at 2023-10-01 11:00:00 for $120

Appointments sorted by consultation fee:
Alice with Dr. Smith at 2023-10-01 09:00:00 for $100
Charlie with Dr. Brown at 2023-10-01 11:00:00 for $120
Bob with Dr. Jones at 2023-10-01 10:00:00 for $150
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Code :

```
week 12 > railway.py > search_ticket
""" Sort bookings based on travel date or seat number. """
4 class Ticket:
5     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
6         self.ticket_id = ticket_id
7         self.passenger_name = passenger_name
8         self.train_number = train_number
9         self.seat_number = seat_number
10        self.travel_date = travel_date
11    def search_ticket(tickets, ticket_id):
12        """
13        Searches for a ticket by its ID.
14
15        Time Complexity: O(n) - where n is the number of tickets.
16        Space Complexity: O(1) - as we are using a constant amount of space for variables.
17
18        Parameters:
19        tickets (list): A list of Ticket objects.
20        ticket_id (str): The ID of the ticket to search for.
21
22        Returns:
23        Ticket: The ticket object if found, otherwise None.
24        """
25        for ticket in tickets:
26            if ticket.ticket_id == ticket_id:
27                return ticket
28        return None
```

```

week 12 > railway.py > search_ticket
29 def sort_tickets(tickets, key):
31     Sorts tickets based on a specified key (either 'travel_date' or 'seat_number').
32
33     Time Complexity: O(n log n) - where n is the number of tickets.
34     Space Complexity: O(1) - as we are using a constant amount of space for variables.
35
36     Parameters:
37     tickets (list): A list of Ticket objects.
38     key (str): The key to sort by ('travel_date' or 'seat_number').
39
40     Returns:
41     list: The sorted list of Ticket objects.
42     """
43     return sorted(tickets, key=lambda x: getattr(x, key))
44
45 # Test cases
46 if __name__ == "__main__":
47     tickets = [
48         Ticket('T001', 'John Doe', 'Train A', '1A', '2024-07-01'),
49         Ticket('T002', 'Jane Doe', 'Train B', '2B', '2024-07-02'),
50         Ticket('T003', 'Alice Smith', 'Train A', '1B', '2024-07-01')
51     ]
52
53     # Test search_ticket
54     print(search_ticket(tickets, 'T002').passenger_name) # Should return 'Jane Doe'
55     print(search_ticket(tickets, 'T004')) # Should return None
56
57     # Test sort_tickets by travel_date
58     sorted_by_date = sort_tickets(tickets, key='travel_date')
59     print("Tickets sorted by travel date:")
60     for ticket in sorted_by_date:
61         print(f"{ticket.passenger_name} - {ticket.travel_date}")
62
63     # Test sort_tickets by seat_number
64     sorted_by_seat = sort_tickets(tickets, key='seat_number')
65     print("Tickets sorted by seat number:")
66     for ticket in sorted_by_seat:
67         print(f"{ticket.passenger_name} - {ticket.seat_number}")

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python313\python.exe .5.py
Found booking: Bob on Train B seat 15B for travel on 2023-10-02

Bookings sorted by travel date:
Alice on Train A seat 12A for travel on 2023-10-01
Charlie on Train C seat 10C for travel on 2023-10-01
Bob on Train B seat 15B for travel on 2023-10-02

Bookings sorted by seat number:
Charlie on Train C seat 10C for travel on 2023-10-01
Alice on Train A seat 12A for travel on 2023-10-01
Bob on Train B seat 15B for travel on 2023-10-02
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```


Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Code :

```
week 12 > 📁 hostel.py > ...
5  class Allocation:
6      def __init__(self, student_id, room_number, floor, allocation_date):
10         self.allocation_date = allocation_date
11  def search_allocation(allocations, student_id):
12      """
13          Searches for allocation details by student ID.
14
15          Time Complexity: O(n) - where n is the number of allocations.
16          Space Complexity: O(1) - as we are using a constant amount of space for variables.
17
18          Parameters:
19          allocations (list): A list of Allocation objects.
20          student_id (str): The ID of the student to search for.
21
22          Returns:
23          Allocation: The allocation object if found, otherwise None.
24      """
25      for allocation in allocations:
26          if allocation.student_id == student_id:
27              return allocation
28      return None

def sort_allocations(allocations, key):
    """Sorts allocations based on a specified key (either 'room_number' or 'allocation_date').

    Time Complexity: O(n log n) - where n is the number of allocations.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
```

```

Parameters:
allocations (list): A list of Allocation objects.
key (str): The key to sort by ('room_number' or 'allocation_date').

Returns:
list: The sorted list of Allocation objects.
"""
return sorted(allocations, key=lambda x: getattr(x, key))

```

```

# Test cases
if __name__ == "__main__":
    allocations = [
        Allocation('S001', '101', '1st Floor', '2024-07-01'),
        Allocation('S002', '102', '1st Floor', '2024-07-02'),
        Allocation('S003', '201', '2nd Floor', '2024-07-01')
    ]

    # Test search_allocation
    print(search_allocation(allocations, 'S002').room_number) # Should return '102'
    print(search_allocation(allocations, 'S004')) # Should return None

    # Test sort_allocations by room_number
    sorted_by_room = sort_allocations(allocations, key='room_number')
    print("Allocations sorted by room number:")
    for allocation in sorted_by_room:
        print(f"{allocation.student_id} - {allocation.room_number}")

    # Test sort_allocations by allocation_date
    sorted_by_date = sort_allocations(allocations, key='allocation_date')
    print("Allocations sorted by allocation date:")
    for allocation in sorted_by_date:
        print(f"{allocation.student_id} - {allocation.allocation_date}")

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python313\python
.5.py"
Found allocation: Student ID 2 in room 102B on 2023-10-02

Allocations sorted by room number:
Student ID 1 in room 101A on 2023-10-01
Student ID 2 in room 102B on 2023-10-02
Student ID 3 in room 201C on 2023-10-01

Allocations sorted by allocation date:
Student ID 1 in room 101A on 2023-10-01
Student ID 3 in room 201C on 2023-10-01
Student ID 2 in room 102B on 2023-10-02
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

Code :

```
week 12 > movie.py > ...
5  class Movie:
6      def __init__(self, movie_id, title, genre, rating, release_year):
7          self.title = title
8          self.genre = genre
9          self.rating = rating
10         self.release_year = release_year
11
12     def search_movie(movies, movie_id):
13         """
14         Searches for a movie by its ID.
15
16         Time Complexity: O(n) - where n is the number of movies.
17         Space Complexity: O(1) - as we are using a constant amount of space for variables.
18
19         Parameters:
20         movies (list): A list of Movie objects.
21         movie_id (str): The ID of the movie to search for.
22
23         Returns:
24         Movie: The movie object if found, otherwise None.
25         """
26         for movie in movies:
27             if movie.movie_id == movie_id:
28                 return movie
29         return None
```

```
def sort_movies(movies, key):
    """
    Sorts movies based on a specified key (either 'rating' or 'release_year').
    Time Complexity: O(n log n) - where n is the number of movies.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    movies (list): A list of Movie objects.
    key (str): The key to sort by ('rating' or 'release_year').
```

```

Returns:
list: The sorted list of Movie objects.
"""

    return sorted(movies, key=lambda x: getattr(x, key))

# Test cases
if __name__ == "__main__":
    movies = [
        Movie('M001', 'Inception', 'Sci-Fi', 8.8, 2010),
        Movie('M002', 'The Godfather', 'Crime', 9.2, 1972),
        Movie('M003', 'The Dark Knight', 'Action', 9.0, 2008)
    ]

    # Test search_movie
    print(search_movie(movies, 'M002').title) # Should return 'The Godfather'
    print(search_movie(movies, 'M004')) # Should return None

    # Test sort_movies by rating
    sorted_by_rating = sort_movies(movies, key='rating')
    print("Movies sorted by rating:")
    for movie in sorted_by_rating:
        print(f"{movie.title} - {movie.rating}")

    # Test sort_movies by release_year
    sorted_by_year = sort_movies(movies, key='release_year')
    print("Movies sorted by release year:")

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python38\python.exe .5.py
Found movie: Movie B (Comedy) with rating 7.2 released in 2019

Movies sorted by rating:
Movie C (Drama) with rating 9.0 released in 2021
Movie A (Action) with rating 8.5 released in 2020
Movie B (Comedy) with rating 7.2 released in 2019

Movies sorted by release year:
Movie B (Comedy) with rating 7.2 released in 2019
Movie A (Action) with rating 8.5 released in 2020
Movie C (Drama) with rating 9.0 released in 2021
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Code :

```
week 12 > agri.py > search_crop
5 class Crop:
6     def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
7         self.crop_name = crop_name
8         self.soil_moisture = soil_moisture
9         self.temperature = temperature
10        self.yield_estimate = yield_estimate
11
12    def search_crop(crops, crop_id):
13        """
14        Searches for crop details by crop ID.
15
16        Time Complexity: O(n) - where n is the number of crops.
17        Space Complexity: O(1) - as we are using a constant amount of space for variables.
18
19        Parameters:
20        crops (list): A list of Crop objects.
21        crop_id (str): The ID of the crop to search for.
22
23        Returns:
24        Crop: The crop object if found, otherwise None.
25        """
26        for crop in crops:
27            if crop.crop_id == crop_id:
28                return crop
29        return None
30
31    def sort_crops(crops, key):
32        """Sorts crops based on a specified key (either 'soil_moisture' or 'yield_estimate').
33
34        Time Complexity: O(n log n) - where n is the number of crops.
35        Space Complexity: O(1) - as we are using a constant amount of space for variables.
36
37        Parameters:
38        crops (list): A list of Crop objects.
39        key (str): The key to sort by ('soil_moisture' or 'yield_estimate').
```

```

Returns:
list: The sorted list of Crop objects.
"""
    return sorted(crops, key=lambda x: getattr(x, key))
# Test cases
if __name__ == "__main__":
    crops = [
        Crop('C001', 'Wheat', 30, 25, 1000),
        Crop('C002', 'Corn', 40, 28, 1500),
        Crop('C003', 'Rice', 35, 30, 1200)
    ]

    # Test search_crop
    print(search_crop(crops, 'C002').crop_name) # Should return 'Corn'
    print(search_crop(crops, 'C004')) # Should return None

    # Test sort_crops by soil_moisture
    sorted_by_moisture = sort_crops(crops, key='soil_moisture')
    print("Crops sorted by soil moisture:")
    for crop in sorted_by_moisture:
        print(f"{crop.crop_name} - {crop.soil_moisture}")

    # Test sort_crops by yield_estimate
    sorted_by_yield = sort_crops(crops, key='yield_estimate')
    print("Crops sorted by yield estimate:")

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments\ai_assignment_5.py
Found crop: Corn with soil moisture 40%, temperature 28°C, and yield estimate 1500 kg

Crops sorted by soil moisture:
Wheat with soil moisture 30%, temperature 25°C, and yield estimate 1000 kg
Rice with soil moisture 35%, temperature 22°C, and yield estimate 1200 kg
Corn with soil moisture 40%, temperature 28°C, and yield estimate 1500 kg

Crops sorted by yield estimate:
Corn with soil moisture 40%, temperature 28°C, and yield estimate 1500 kg
Rice with soil moisture 35%, temperature 22°C, and yield estimate 1200 kg
Wheat with soil moisture 30%, temperature 25°C, and yield estimate 1000 kg
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Code :

```
week 12 > flight.py > ...
5  class Flight:
6      def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
7
8
9      self.departure_time = departure_time
10     self.arrival_time = arrival_time
11     self.status = status
12
13     def search_flight(flights, flight_id):
14         """
15         Searches for flight details by flight ID.
16
17         Time Complexity: O(n) - where n is the number of flights.
18         Space Complexity: O(1) - as we are using a constant amount of space for variables.
19
20         Parameters:
21         flights (list): A list of Flight objects.
22         flight_id (str): The ID of the flight to search for.
23
24         Returns:
25         Flight: The flight object if found, otherwise None.
26         """
27         for flight in flights:
28             if flight.flight_id == flight_id:
29                 return flight
30         return None
```

```

def sort_flights(flights, key):
    """Sorts flights based on a specified key (either 'departure_time' or 'arrival_time').

    Time Complexity: O(n log n) - where n is the number of flights.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    flights (list): A list of Flight objects.
    key (str): The key to sort by ('departure_time' or 'arrival_time').

    Returns:
    list: The sorted list of Flight objects.
    """
    return sorted(flights, key=lambda x: getattr(x, key))

# Test cases
if __name__ == "__main__":
    flights = [
        Flight('F001', 'Airline A', '2024-07-01 10:00', '2024-07-01 12:00', 'On Time'),
        Flight('F002', 'Airline B', '2024-07-01 11:00', '2024-07-01 13:00', 'Delayed'),
        Flight('F003', 'Airline C', '2024-07-01 09:00', '2024-07-01 11:00', 'On Time')
    ]

    # Test search_flight
    print(search_flight(flights, 'F002').airline_name) # Should return 'Airline B'
    print(search_flight(flights, 'F004')) # Should return None

    # Test sort_flights by departure_time
    sorted_by_departure = sort_flights(flights, key='departure_time')
    print("Flights sorted by departure time:")
    for flight in sorted_by_departure:
        print(f"{flight.flight_id} - {flight.departure_time}")

    # Test sort_flights by arrival_time
    sorted_by_arrival = sort_flights(flights, key='arrival_time')
    print("Flights sorted by arrival time:")

```

Output :

```

PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments> & C:\Users\hasin\AppData\Local\Programs\Python\Python313\python.exe .5.py
Found flight: Airline B departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status Delayed

Flights sorted by departure time:
Airline C departing at 2023-10-01 07:00 arriving at 2023-10-01 09:00 with status On Time
Airline A departing at 2023-10-01 08:00 arriving at 2023-10-01 10:00 with status On Time
Airline B departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status Delayed

Flights sorted by arrival time:
Airline C departing at 2023-10-01 07:00 arriving at 2023-10-01 09:00 with status On Time
Airline A departing at 2023-10-01 08:00 arriving at 2023-10-01 10:00 with status On Time
Airline B departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status Delayed
PS C:\Users\hasin\OneDrive\Desktop\java programming\ai assignments>

```


